

Câu 1 — Reattach an Orphaned PersistentVolume

Đề bài

A user accidentally deleted the Redis Deployment in the `redis-store` namespace. The Deployment was configured with persistent storage. Your job is to restore the Deployment while ensuring data is preserved by reusing the available PersistentVolume.

Tasks:

- Only one PersistentVolume exists in the cluster, currently `Released` and available for reuse.
- Create a PersistentVolumeClaim named `redis-data` in the `redis-store` namespace with access mode `ReadWriteOnce` and storage request `500Mi`.
- Edit the Redis Deployment manifest at `~/redis-deploy.yaml` to reference the PVC you just created.
- Apply the updated manifest.
- Confirm the Redis Deployment is running and stable.
- Hoặc yêu cầu tạo storage class, config storage class đó thành default storage class sau đó tạo pvc sử dụng sc đó và mount pvc cho pod

Lời giải

Bước 1 - Xem trạng thái PV hiện có

```
k get pv
```

```
k describe pv <pv-name>
```

Vì sao: cần biết PV đang ở trạng thái gì (`Released` hay `Available`), capacity và `storageClassName` của nó. Nếu PV đang `Released` (do PVC cũ đã bị xóa), PV sẽ giữ nguyên `claimRef` cũ và **không tự động** cho PVC mới bind vào cho tới khi mình gỡ `claimRef` đó ra.

 <https://kubernetes.io/docs/concepts/storage/persistent-volumes/#reclaiming>

Bước 2 - Gỡ `claimRef` cũ nếu PV đang ở trạng thái `Released`

```
k patch pv <pv-name> -p '{"spec":{"claimRef": null}}'
```

(hoặc `edit pv <pv-name>` rồi xóa phần `claimRef` cũ đi)

Đây là bước hay bị quên nhất - nếu bỏ qua, PVC mới tạo ra sẽ nằm ở trạng thái `Pending` mãi vì PV vẫn "nhớ" nó thuộc về PVC cũ đã không còn tồn tại. Cần đảm bảo PV ở trạng thái `Available` trước khi tạo PVC mới.

Bước 3 - Tạo PVC mới khớp với PV

 <https://kubernetes.io/docs/concepts/storage/persistent-volumes/#persistentvolumeclaims>

 **YAML**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: redis-data
  namespace: redis-store

spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 500Mi

  storageClassName: <storage-class-name>
```

Áp dụng:

 **Bash**

```
kubectl apply -f pvc.yaml
```

Không set `storageClassName` khác với PV (để trống nếu PV cũng không set) - đây là cách để Kubernetes tự match PVC với đúng PV cũ theo `accessMode` + `capacity`, thay vì tạo volume hoàn toàn mới.

 <https://kubernetes.io/docs/concepts/storage/persistent-volumes/#binding>

Bước 4 - Trỏ Deployment vào PVC

Sửa ~/redis-deploy.yaml:

 **YAML** 

```
volumes:
  - name: redis-storage
    persistentVolumeClaim:
      claimName: redis-data
```

Vì sao: Pod chỉ đọc được dữ liệu cũ nếu volume trong Deployment tham chiếu đúng đến PVC vừa tạo (chứ không phải `emptyDir` hay PVC khác).

 <https://kubernetes.io/docs/concepts/storage/volumes/#persistentvolumeclaim>

Bước 5 - Apply lại Deployment

```
k apply -f ~/redis-deploy.yaml
```

Hướng dẫn kiểm tra lại

```
k get pvc redis-data -n redis-store # STATUS: Bound
```

```
k get pv # STATUS: Bound, CLAIM: redis-store/redis-data
```

```
k rollout status deployment/redis -n redis-store
```

```
k get pods -n redis-store # Running, READY 1/1
```

Câu 2 — Argo CD via Helm Template Without CRDs

Đề bài

Install Argo CD in a Kubernetes cluster using Helm while ensuring the CRDs are **not** installed (they are already pre-installed in the cluster).

Tasks:

- Add the official Argo CD Helm repository with the name `argo-helm` (<https://argoproj.github.io/argo-helm>).
- Create a namespace called `gitops-ns`.
- Generate a Helm template from the Argo CD chart, version 7.9.0, for the `gitops-ns` namespace.
- Ensure CRDs are not rendered by the template.
- Save the generated manifest to `/root/argocd-template.yaml`.
- Install Argo CD using Helm with release name `argocd`, using the **same chart version** (7.9.0) and the **same configuration** as the template above (CRDs disabled), into the `gitops-ns` namespace.

Lời giải

Bước 1 - Tạo namespace trước

```
k create namespace gitops-ns
```

Bước 2 - Add repo Helm và refresh index

```
helm repo add argo-helm https://argoproj.github.io/argo-helm
```

```
helm repo update
```

`helm repo update` đảm bảo local index có đúng version 7.9.0 mới nhất, tránh lỗi "chart version not found" nếu index đang cũ.

 https://helm.sh/docs/helm/helm_repo_add/

Bước 3 - Render template với CRDs tắt, lưu ra file

```
helm template argocd argo-helm/argo-cd --version 7.9.0 \  
--set crds.install=false --namespace gitops-ns > /root/argocd-template.yaml
```

- flag `crds.install=false` là flag chuẩn của chart `argo-cd` để loại CRD khỏi output (nhiều bạn bị nhầm với `--skip-crds`) - vì đề nói CRDs đã có sẵn, apply thêm CRD từ chart có thể gây version conflict.

 https://helm.sh/docs/helm/helm_template/

 <https://argoproj.github.io/argo-helm> (link này không được dùng trong khi thi CKA)

Bước 4 - Install release bằng đúng version và cấu hình đã dùng ở template

```
helm install argocd argo-helm/argo-cd --version 7.9.0 \
```

```
--set crds.install=false --namespace gitops-ns
```

`helm template` chỉ render YAML ra để xem trước (dry-run), **không** thực sự tạo resource trong cluster - đề bài yêu cầu thêm bước `helm install` mới thật sự deploy Argo CD. Phải giữ đúng `--version` và `--set crds.install=false` giống hết lúc template để đảm bảo cấu hình cài đặt thật khớp với bản đã review, tránh trường hợp "template 1 kiểu, cài 1 kiểu".  https://helm.sh/docs/helm/helm_install/

Hướng dẫn kiểm tra lại

```
helm repo list | grep argo-helm
```

```
k get ns gitops-ns
```

```
ls -l /root/argocd-template.yaml # file tồn tại, không rỗng
```

```
grep -c "kind: CustomResourceDefinition" /root/argocd-template.yaml # kết quả phải là 0
```

```
helm list -n gitops-ns # release "argocd", CHART argo-cd-7.9.0, STATUS deployed
```

```
k get pods -n gitops-ns # các pod argocd đang khởi động/Running
```

```
k get crd | grep argoproj.io # KHÔNG có CRD nào của argoproj.io được cài thêm
```

Câu 3 — Sidecar Container Sharing a Log Volume

Đề bài

Update the existing `apache-web` Deployment (namespace `default`) by adding a sidecar container to the existing Pod template.

Tasks:

- Add a sidecar container named `log-shipper` using the `busybox:stable` image.
- The sidecar must run: `/bin/sh -c "tail -f /var/log/apache-web.log"`.
- Use a shared volume mounted at `/var/log` so the sidecar can read the log file written by the main container.

Lời giải

Bước 1 - Thêm volume dùng chung và container sidecar

 **YAML** 

```
cat <<'EOF' | k apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-web
spec:
  template:
    spec:
      volumes:
        - name: log
          emptyDir: {}

      containers:
        - name: apache-web
          volumeMounts:
            - name: log
              mountPath: /var/log

        - name: log-shipper
          image: busybox:stable
          command:
            - /bin/sh
            - -c
            - tail -f /var/log/apache-web.log
          volumeMounts:
            - name: log
              mountPath: /var/log
```

EOF



Cả 2 container cùng mount 1 `emptyDir` tại `/var/log` nên container chính ghi log ra đâu, sidecar đọc được ngay đó - đây chính là pattern "sidecar container for logging" chuẩn của Kubernetes.



<https://kubernetes.io/docs/concepts/cluster-administration/logging/#using-a-sidecar-container-with-the-logging-agent> (template ở gần cuối)



<https://kubernetes.io/docs/concepts/workloads/pods/#how-pods-manage-multiple-containers>

Bước 2 - Chờ rollout hoàn tất

```
k rollout status deployment apache-web
```

Sửa Pod template sẽ trigger rolling update - phải đợi Pod mới (2 container) thay thế Pod cũ (1 container) xong mới kiểm tra được.

Hướng dẫn kiểm tra lại

```
k get pods -l app=apache-web # READY 2/2
```

```
k logs deploy/apache-web -c log-shipper --tail=5 # thấy log stream từ container chính
```

Câu 4 — Fair Resource Allocation Across Pods

Đề bài

You are managing a "blog-app" Deployment (5 replicas, namespace `eager-fox`) in a Kubernetes cluster. Adjust the Pod resource requests and limits to ensure stable node operation.

Tasks:

- Scale the `blog-app` Deployment down to 0 replicas.
- Divide the node's allocatable resources evenly across all 5 pods.
- Assign fair and equal CPU/memory to each Pod.
- Add enough overhead so the node doesn't become unstable.
- Init containers and main containers must use identical resource requests and limits.
- Scale the Deployment back to 5 replicas.

Lời giải

Bước 1 - Tạm dừng workload để sửa an toàn

```
k scale deployment blog-app -n eager-fox --replicas 0
```

Vì sao: sửa resources của container đang chạy sẽ trigger restart Pod ngay lập tức và có thể gây gián đoạn tạm thời nếu sửa từng pod một; scale về 0 rồi sửa 1 lần ở Deployment level rồi scale lại sẽ gọn và an toàn hơn.

Bước 2 - Xem allocatable thật của node, tính toán chia đều

```
k describe node <node-name> | grep -A5 Allocatable
```

Ví dụ node có `cpu: 2`, `memory: 4Gi`. Chừa lại ~20% làm overhead cho hệ thống (kube-system, OS):

```
awk 'BEGIN {print 2 * 0.8 / 5}' # CPU/pod ≈ 0.32 -> chọn 300m
```

```
awk 'BEGIN {print 4096 * 0.8 / 5}' # Mem/pod ≈ 655 -> chọn 600Mi
```

Đề chấm điểm theo tinh thần "chia đều + có headroom", không có 1 con số cứng - miễn tổng requests của 5 pod không vượt quá phần allocatable đã trừ overhead là đạt.

 <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/#requests-and-limits>

Bước 3 - Set giống nhau cho cả init container và container chính

```
k edit deployment blog-app -n eager-fox
```


Thêm block sau vào **cả** `spec.template.spec.initContainers[]` lẫn `spec.template.spec.containers[]`:

 **YAML** 

```
resources:
  requests:
    cpu: "300m"
    memory: "600Mi"
  limits:
    cpu: "400m"
    memory: "700Mi"
```

Vì sao: đề yêu cầu rõ init container và main container phải giống hệt nhau - đây là điểm hay bị bỏ sót vì mặc định người ta chỉ sửa container chính.

Bước 4 - Bật lại Deployment

```
k scale deployment blog-app -n eager-fox --replicas 5
```

Hướng dẫn kiểm tra lại

```
k rollout status deployment blog-app -n eager-fox
```

```
k get pods -n eager-fox -l app=blog-app # 5 pod Running
```

```
k describe deployment blog-app -n eager-fox
```

(Đảm bảo `.spec.template.spec.initContainers[*].resources` & `spec.template.spec.containers[*].resources` giống nhau)

Câu 5 — Horizontal Pod Autoscaler with Stabilization Window

Đề bài

Create a new HorizontalPodAutoscaler named `web-api` in the `scaling-lab` namespace.

Tasks:

- Target the existing Deployment `web-api-deployment` in `scaling-lab`.
- Target 60% average CPU utilization per Pod.
- Minimum 2 pods, maximum 5 pods.
- Set the downscale stabilization window to 45 seconds.

Lời giải

Bước 1 - Xác nhận Deployment target tồn tại

```
k get deploy web-api-deployment -n scaling-lab
```

Bước 2 - Viết manifest HPA (không dùng `kubectl autoscale` vì lệnh này không set được `behavior`)

</> YAML



```
cat <<'EOF' > hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-api
  namespace: scaling-lab

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-api-deployment

  minReplicas: 2
  maxReplicas: 5

  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60

  behavior:
    scaleDown:
      stabilizationWindowSeconds: 45
EOF
```

```
k apply -f hpa.yaml
```

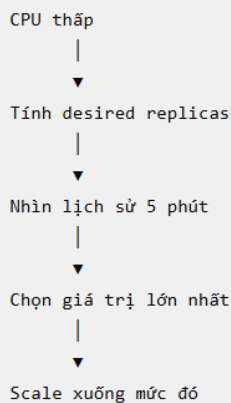
- field `behavior.scaleDown.stabilizationWindowSeconds` **chỉ tồn tại ở `autoscaling/v2`**, không có ở `v1`. Stabilization window dùng để tránh HPA "flapping" (scale up/down liên tục) khi metric dao động, bằng cách nhìn lại state mong muốn trong khoảng thời gian gần nhất trước khi thực sự scale down.



<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/#autoscaling-on-multiple-metrics-and-custom-metrics>

<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/#stabilization-window>

Scale Down



```
graph TD; A[CPU thấp] --> B[Tính desired replicas]; B --> C[Nhìn lịch sử 5 phút]; C --> D[Chọn giá trị lớn nhất]; D --> E[Scale xuống mức đó];
```

CPU thấp

▼

Tính desired replicas

▼

Nhìn lịch sử 5 phút

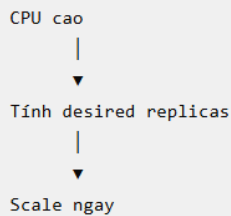
▼

Chọn giá trị lớn nhất

▼

Scale xuống mức đó

Scale Up



```
graph TD; A[CPU cao] --> B[Tính desired replicas]; B --> C[Scale ngay];
```

CPU cao

▼

Tính desired replicas

▼

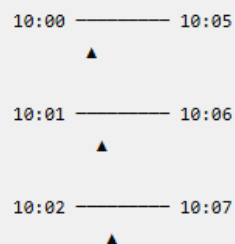
Scale ngay

`stabilizationWindowSeconds` không trì hoãn việc scale, mà yêu cầu HPA tham khảo lịch sử các **desired replicas trong khoảng thời gian đó**. Với `scaleDown`, HPA chọn **desired replica lớn nhất** trong cửa sổ để tránh giảm Pod quá sớm và gây hiện tượng **flapping** (liên tục scale xuống rồi scale lên).

Vì sao tài liệu gọi là "rolling maximum"?

Giả sử cửa sổ là 5 phút.

Ở mỗi thời điểm, HPA luôn nhìn **5 phút gần nhất**, nên cửa sổ này liên tục "trượt" theo thời gian:



10:00 ————— 10:05

▲

10:01 ————— 10:06

▲

10:02 ————— 10:07

▲

Mỗi lần đánh giá, nó lấy **giá trị lớn nhất trong cửa sổ hiện tại**. Vì vậy tài liệu gọi đây là **rolling maximum** (giá trị lớn nhất trong một cửa sổ thời gian trượt).

Hướng dẫn kiểm tra lại

```
k get hpa web-api -n scaling-lab
```

```
# REFERENCE: Deployment/web-api-deployment, MINPODS 2, MAXPODS 5, TARGETS cpu:  
<x>%/60%
```

```
k get hpa web-api -n scaling-lab -o yaml | grep -A2 scaleDown
```

```
# stabilizationWindowSeconds: 45
```

Câu 6 — Custom Resource Definitions (cert-manager)

Đề bài

Verify the cert-manager CRDs installed in the cluster.

Tasks:

- List all cert-manager CRDs and save the output to `/root/cm-crds.yaml` using `kubectl`'s default output format.
- Extract the documentation for the `dnsNames` field of the `Certificate` custom resource and save it to `/root/dnsnames.yaml`. Any output format is allowed for this part.

Lời giải

Bước 1 - Liệt kê CRD của cert-manager

```
k get crd | grep cert-manager > /root/cm-crds.yaml
```

Đề yêu cầu **default output format**, tức là không được thêm `-o yaml`/`-o json` hay vào file xóa thời gian để chỉ giữ lại mỗi tên crds - dùng đúng format bảng mặc định của `k get`. `grep` chỉ để lọc đúng các CRD liên quan tới cert-manager trong hàng trăm CRD có thể có trong cluster.

 <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>

Bước 2 - Explain field dnsNames

```
k explain certificate.spec.dnsNames > /root/dnsnames.yaml
```

Duy nhất chỉ có `kubectl explain` đọc trực tiếp OpenAPI schema mà CRD đăng ký với API server. (Nhiều bạn nhầm với `kubectl describe` hoặc `kubectl get -o yaml`)

 <https://kubernetes.io/docs/reference/kubectl/>

Hướng dẫn kiểm tra lại

```
cat /root/cm-crds.yaml # liệt kê certificates.cert-manager.io,  
issuers.cert-manager.io, ... kèm thời gian
```

```
cat /root/dnsnames.yaml # có mô tả field "dnsNames" (danh sách tên miền cho  
chứng chỉ)
```

Câu 7 — PriorityClass for Critical Workloads

Đề bài

There is an existing Deployment `log-collector` in the `workloads` namespace. The cluster already has at least one user-defined `PriorityClass`.

Tasks:

- Create a new `PriorityClass` named `elevated-priority` for user workloads. Its value must be exactly one less than the highest existing user-defined `PriorityClass` value.
- Patch `log-collector` in `workloads` to use `elevated-priority`.

Lời giải

Bước 1 - Tìm giá trị priority cao nhất hiện có (bỏ qua system classes)

```
k get priorityclass
```

Vì sao: các class `system-cluster-critical` và `system-node-critical` là built-in, không tính vào "user-defined" - phải loại trừ 2 class này khi tìm giá trị cao nhất.

 <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/#priorityclass>

Bước 2 - Tạo PriorityClass mới = giá trị cao nhất - 1

Giả sử class user-defined cao nhất tên `top-tier` có value: 900000

-> **priority** cần tạo có value: $900000 - 1 = 899999$

```
k create priorityclass elevated-priority --value=899999
```

Bước 3 - Patch Deployment dùng class mới

```
k edit deployment log-collector -n workloads
```

- Bổ sung **priorityClassName**: `elevated-priority`

vào `spec.template.spec` (Pod template) chứ không phải spec gốc của Deployment - vì `priorityClassName` là thuộc tính của Pod, không phải của Deployment.



<https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/#how-to-use-priority-and-preemption>



<https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/#pod-priority>

Hướng dẫn kiểm tra lại

```
k get priorityclass elevated-priority # VALUE = 899999 (đúng bằng cao nhất - 1)
```

```
k -n workloads describe deploy log-collector | grep priorityClassName
```

```
# elevated-priority, pod đã được tạo lại và Running
```


Câu 8 — Install a CNI That Enforces Network Policy

Đề bài

Install and configure a CNI that meets the following requirements, from a manifest (not Helm). Choose one of: Flannel v0.27.0 (manifest kube-flannel.yml) or Calico v3.29.1 (manifest tigera-operator.yml).

Tasks:

- Pods must be able to communicate with each other across nodes.
- The CNI must enforce Kubernetes NetworkPolicy.
- Must be installed via manifest, not Helm.

Lời giải

Bước 1 - Chọn CNI đúng yêu cầu

Flannel **không** hỗ trợ enforce NetworkPolicy (chỉ lo phần network overlay) - nên chỉ Calico thỏa cả 2 điều kiện.

 <https://kubernetes.io/docs/concepts/services-networking/network-policies/>



<https://kubernetes.io/docs/concepts/cluster-administration/networking/#how-to-implement-the-kubernetes-network-model>

Bước 2 - Cài Calico từ manifest bằng kubectl create (không dùng apply)

```
k create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.29.1/manifests/tigera-operator.yml
```

Vì sao: manifest tigera-operator.yml chứa CRD rất lớn, dễ vượt giới hạn kích thước annotation kubectl.kubernetes.io/last-applied-configuration mà kubectl apply tự thêm vào - dùng create để tránh lỗi "metadata.annotations: Too long".

Lưu ý: bước này **chỉ** cài operator + CRD, Calico **chưa thực sự chạy** cho tới khi có bước 5 bên dưới.

 Docs chính thức của Calico (sẽ được cập trong đề thi CKA)

Bước 3 - Kiểm tra pod CIDR thực tế của cluster (rất hay bị bỏ quên)

```
k cluster-info dump | grep -m 1 cluster-cidr
```

Vì sao: file `custom-resources.yaml` mặc định của Calico luôn set CIDR là `192.168.0.0/16`. Nếu cluster của mình được khởi tạo với `--pod-network-cidr` khác (ví dụ `10.244.0.0/16`), mà cứ apply nguyên bản mặc định thì Calico sẽ cấp IP cho Pod nằm ngoài dải PodCIDR thật của từng node → Pod bị lỗi network hoặc không chạy được, y hệt lỗi lệch CIDR thường gặp ở bài Flannel. Đây là bước nhiều người thì quên nhất vì `tigera-operator.yaml` apply xong nhìn có vẻ "chạy được" nhưng thực chất Calico chưa hề khởi tạo network.

***Bước 4 (optional) - Tải `custom-resources.yaml` về và sửa CIDR nếu khác mặc định

`curl -O https://raw.githubusercontent.com/projectcalico/calico/v3.29.1/manifests/custom-resources.yaml`

Nếu CIDR ở bước 3 khác `192.168.0.0/16`, sửa lại đúng phần `spec.calicoNetwork.ipPools[].cidr` trong file vừa tải:

```
</> YAML
spec:
  calicoNetwork:
    ipPools:
      - name: default-ipv4-ippool
        blockSize: 26
        cidr: 10.244.0.0/16 # Đổi cho khớp với Pod CIDR thật của cluster
        encapsulation: VXLANCrossSubnet
```

Vì sao: đây chính là nơi Calico "biết" phải phát IP theo dải nào cho Pod - nếu để mặc định mà cluster dùng CIDR khác, coi như cấu hình sai ngay từ đầu dù các bước sau làm đúng hết.



<https://kubernetes.io/docs/concepts/cluster-administration/networking/#how-to-implement-the-kubernetes-network-model>

Bước 5 - Apply `custom-resources.yaml` để Calico thực sự khởi tạo network

`kubectl apply -f custom-resources.yaml`

* Trường hợp CIDR giống mặc định thì chạy luôn:

`kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.29.1/manifests/custom-resources.yaml`

Vì sao: resource `Installation` trong file này mới là thứ khiến `tigera-operator` thực sự tạo ra `DaemonSet` `calico-node`, `calico-kube-controllers`,... trong namespace `calico-system`. Thiếu bước này, operator vẫn "đứng chờ" chứ Calico chưa hoạt động.

<https://kubernetes.io/docs/concepts/services-networking/network-policies/>

Hướng dẫn kiểm tra lại

bash

```
kubectrl get all -n tigera-operator # pod/deployment tigera-operator Running
```

```
kubectrl get pods -n calico-system # calico-node, calico-kube-controllers, calico-typha Running
```

```
kubectrl get tigerastatus # tất cả component AVAILABLE=True
```

```
kubectrl get nodes # tất cả Ready
```

```
kubectrl get pods -A | grep -Ei 'calico|tigera'
```

Câu 9 — Configure containerd and Kernel Parameters

Đề bài

Configure a node so it is ready to join a cluster using `containerd` as the container runtime.

Tasks:

- Install the package `~/runc.deb` using `dpkg`.
- Enable and start the `containerd` service.
- Set `net.bridge.bridge-nf-call-iptables` to 1.
- Set `net.bridge.bridge-nf-call-ip6tables` to 1.
- Set `net.ipv4.ip_forward` to 1.
- Set `net.netfilter.nf_conntrack_max` to 262144.

Lời giải

Bước 1 - Cài package và bật service

```
sudo dpkg -i ~/runc.deb
```

```
sudo systemctl enable --now containerd
```

`enable --now` vừa bật service ngay lập tức vừa đảm bảo tự start lại sau khi reboot - tiết kiệm 1 lệnh so với chạy `enable` rồi `start` riêng. (Bạn nào thích dùng 2 lệnh `start` & `enable` riêng cũng không sao)

 <https://kubernetes.io/docs/setup/production-environment/container-runtimes/>

Bước 2 - Set sysctl bền vững qua reboot

```
sudo tee /etc/sysctl.d/k8s.conf >/dev/null <<'EOF'
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
net.netfilter.nf_conntrack_max=262144
EOF

sudo sysctl --system
```

Vì sao: set trực tiếp bằng `sysctl -w` chỉ có hiệu lực tới lần reboot kế tiếp; ghi vào `/etc/sysctl.d/` rồi chạy `sysctl --system` mới đảm bảo cấu hình giữ nguyên sau khi node restart - đây là điểm hay bị nhầm sai nếu chỉ set tạm thời.



<https://kubernetes.io/docs/setup/production-environment/container-runtimes/#prerequisite-ipv4-forwarding-optional>

Hướng dẫn kiểm tra lại

```
sudo systemctl status containerd
# Kết quả mong đợi: active (running)

sysctl \
  net.bridge.bridge-nf-call-iptables \
  net.bridge.bridge-nf-call-ip6tables \
  net.ipv4.ip_forward \
  net.netfilter.nf_conntrack_max

# Kết quả mong đợi:
# net.bridge.bridge-nf-call-iptables = 1
# net.bridge.bridge-nf-call-ip6tables = 1
# net.ipv4.ip_forward = 1
# net.netfilter.nf_conntrack_max = 262144
```

Câu 10 — Migrate Ingress to Gateway API

Đề bài

You have an existing web application exposed via an Ingress named `api-ingress`. Migrate it to the Gateway API while keeping the existing HTTPS configuration. A GatewayClass named `nginx-gw` is already installed.

Tasks:

- Create a Gateway named `api-gateway` with hostname `api.gateway.local`, preserving the TLS/listener config from `api-ingress`.
- Create an HTTPRoute named `api-route` with hostname `api.gateway.local`, preserving the routing rules from `api-ingress`.

Lời giải

Bước 1 - Xem cấu hình Ingress hiện tại để lấy đúng thông tin cần giữ lại

```
k describe ingress api-ingress
```

```
# host: api.gateway.local, TLS secret: api-tls, backend: api-service:80, path:  
/ (Prefix)
```

Vi phải chuyển đúng host, tên Secret TLS và backend service/port sang Gateway API, thiếu bất kỳ phần nào cũng khiến traffic không chạy đúng như cũ.

 <https://gateway-api.sigs.k8s.io/guides/getting-started/migrating-from-ingress/>

Bước 2 - Tạo Gateway (giữ nguyên host + TLS)

 **YAML**



```
cat <<'EOF' > gw.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: api-gateway

spec:
  gatewayClassName: nginx-gw

  listeners:
  - name: https
    protocol: HTTPS
    port: 443
    hostname: api.gateway.local
    tls:
      mode: Terminate
      certificateRefs:
      - kind: Secret
        name: api-tls

EOF
```

```
k apply -f gw.yaml
```

 <https://gateway-api.sigs.k8s.io/guides/user-guides/tls/#listeners-with-different-certificates>

Bước 3 - Tạo HTTPRoute (giữ nguyên routing rule)

</> YAML



```
cat <<'EOF' > http.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route

spec:
  parentRefs:
    - name: api-gateway

  hostnames:
    - "api.gateway.local"

  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /

      backendRefs:
        - name: api-service
          port: 80
EOF
```

```
k apply -f http.yaml
```

Vì sao: parentRefs là cách HTTPRoute "gắn" vào đúng Gateway đã tạo ở bước 2 - thiếu field này thì route sẽ không có hiệu lực dù tạo thành công.

 <https://gateway-api.sigs.k8s.io/guides/user-guides/http-routing/>

Hướng dẫn kiểm tra lại

```
k describe gateway api-gateway
```

```
# Listener https, port 443, protocol HTTPS, hostname api.gateway.local,
certificateRef api-tls
```

```
k describe httproute api-route
```

```
# parentRef api-gateway, hostname api.gateway.local, rule PathPrefix / ->
api-service:80
```


Câu 11 — Expose a Deployment via NodePort and Ingress

Đề bài

Expose the existing `echo-server` Deployment in namespace `echo-lab`.

Tasks:

- Create a Service named `echo-svc`, type `NodePort`, port `9090`.
- Create an Ingress named `echo-ing` in `echo-lab` routing `http://echo.example.local/ping` to that service.
- Verify with `curl NODEIP:NODEPORT/ping`, and (if an ingress controller + `/etc/hosts` entry are set up) `curl -o /dev/null -s -w "%{http_code}\n" http://echo.example.local/ping`.

Lời giải

Bước 1 - Expose Deployment bằng Service NodePort

```
k expose deployment echo-server -n echo-lab --name echo-svc \
--type NodePort --port 9090 --target-port 9090
```

 <https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport>

Bước 2 - Tạo Ingress trỏ đến Service

YAML

```
cat <<'EOF' > ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: echo-ing
  namespace: echo-lab

spec:
  rules:
  - host: echo.example.local
    http:
      paths:
      - path: /ping
        pathType: Prefix
        backend:
          service:
            name: echo-svc
            port:
              number: 9090
EOF
```

```
k apply -f ingress.yaml
```

pathType: Prefix cho phép match mọi request bắt đầu bằng /ping, đúng tinh thần đề chỉ yêu cầu route 1 path cụ thể.

<https://kubernetes.io/docs/concepts/services-networking/ingress/#the-ingress-resource>

Hướng dẫn kiểm tra lại

```
k get svc echo-svc -n echo-lab # TYPE NodePort, PORT(S) 9090:3XXXX/TCP
```

```
curl http://$NODE_IP:$NODE_PORT/ping # trả về response từ echo-server
```

Câu 12 — Choose the Least-Permissive NetworkPolicy

Đề bài

There are two Deployments: `web-tier` (namespace `web-tier`) and `api-tier` (namespace `api-tier`). Three candidate NetworkPolicy files exist in `/root/netpols/`.

Tasks:

- Review the 3 candidate policies.
- Apply the one that allows `web-tier` to reach `api-tier` on TCP port 8080 in the **least permissive** way possible.

Lời giải

Bước 1 - Đọc kỹ cả 3 file để so sánh phạm vi cho phép

```
cat /root/netpols/policy-1.yaml
```

```
cat /root/netpols/policy-2.yaml
```

```
cat /root/netpols/policy-3.yaml
```

Ví dụ khi phân tích:

- Policy 1: `podSelector: {}` + ingress rule trống → cho phép **toàn bộ** traffic vào mọi pod trong namespace `api-tier`. Quá rộng.
- Policy 2: đúng namespace `web-tier` nhưng cộng thêm cả 1 dải CIDR khác (`10.0.0.0/8`) → thừa quyền không cần thiết.
- Policy 3: chỉ target pod `app=api-tier`, chỉ cho phép từ namespace `web-tier` + pod `app=web-tier`, chỉ port 8080. → đây là đáp án đúng (ít quyền nhất nhưng vẫn đủ để hoạt động).

Vì sao phải phân tích kỹ: NetworkPolicy càng hẹp càng an toàn (nguyên tắc least privilege) - đề CKA thường test đúng khả năng đọc hiểu YAML để chọn cấu hình chặt nhất chứ không phải cấu hình "chạy được là xong".

 <https://kubernetes.io/docs/concepts/services-networking/network-policies/>

Bước 2 - Xác nhận label trước khi apply

```
k get pods -n web-tier --show-labels # confirm app=web-tier
```

Bước 3 - Apply đúng policy đã chọn

```
k apply -f /root/netpols/policy-3.yaml
```

Hướng dẫn kiểm tra lại

```
k get networkpolicy -n api-tier
```

```
k exec -n web-tier deploy/web-tier -- \
```

```
curl -s --max-time 5 api-tier-service.api-tier.svc.cluster.local:8080
```

```
# Kết nối thành công (cho thấy policy vẫn cho phép đúng traffic cần thiết)
```

Câu 13 — Create and Switch the Default StorageClass

Đề bài

Manage StorageClasses without touching existing workloads.

Tasks:

- Create a StorageClass named `local-fast` using provisioner `rancher.io/local-path`, `volumeBindingMode: WaitForFirstConsumer`. Do **not** mark it default at creation.
- Patch it afterward to become the default StorageClass.
- Ensure `local-fast` is the only default class (the existing default is named `standard`).
- Do not modify any existing Deployment or PVC.

Lời giải

Bước 1 - Tạo StorageClass, không đặt default

</> YAML

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-fast

provisioner: rancher.io/local-path
volumeBindingMode: WaitForFirstConsumer
```

 <https://kubernetes.io/docs/concepts/storage/storage-classes/#storageclass-objects>

Bước 2 - Patch để trở thành default (đúng như đề yêu cầu)

```
k patch storageclass local-fast -p
'{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```



<https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/#changing-the-default-storageclass>

Bước 3 - Bỏ default ở StorageClass cũ (nếu không có StorageClass cũ thì không cần)

```
k patch storageclass standard \
-p
'{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```



<https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/#changing-the-default-storageclass>

Vì sao: Kubernetes cho phép nhiều StorageClass cùng đánh dấu default cùng lúc (dù không nên) - nếu để cả 2 default, PVC không chỉ định storageClassName sẽ dùng class được tạo **sau cùng** theo timestamp, gây khó đoán. Đề yêu cầu rõ chỉ 1 default duy nhất.

Hướng dẫn kiểm tra lại

```
k get storageclass
# local-fast (default) rancher.io/local-path ... WaitForFirstConsumer
# standard ... (không còn chữ "default")
```

Câu 14 — Fix a Broken kube-apiserver → etcd Connection

Đề bài

After migrating the control-plane node to new hardware, kube-apiserver fails to start. Before the migration, etcd was external; after migration, the --etcd-servers flag was accidentally pointed at the wrong IP address (the old node's IP instead of the new etcd endpoint).

Tasks:

- Diagnose why kube-apiserver is failing.
- Correct the etcd endpoint so the API server starts successfully again.

Lời giải

Bước 1 - Chẩn đoán

kubectl sẽ không chạy được vì chính API server đang down, nên phải soi trực tiếp qua container runtime và static pod manifest:

```
k get nodes # timeout / connection refused

sudo crictl ps -a | grep kube-apiserver # container đang restart liên tục

sudo grep etcd-servers /etc/kubernetes/manifests/kube-apiserver.yaml

# --etcd-servers=https://192.168.1.10:2379 <- sai, IP etcd cũ đã đổi sang 192.168.1.20
```

Vì sao: khi API server down, kubectl hoàn toàn vô dụng - phải dùng crictl (giao tiếp trực tiếp với container runtime) và đọc thẳng file manifest tĩnh để chẩn đoán, đây là kỹ năng troubleshooting cốt lõi của CKA.

 <https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/>

 <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/troubleshooting-kubeadm/>

Bước 2 - Sửa đúng IP etcd, lưu file

```
sudo vi /etc/kubernetes/manifests/kube-apiserver.yaml

# đổi: --etcd-servers=https://192.168.1.10:2379

# thành: --etcd-servers=https://192.168.1.20:2379
```

Vì sao: static pod manifest được kubelet theo dõi liên tục (watch) - chỉ cần lưu file là kubelet tự động xóa và tạo lại Pod kube-apiserver mà không cần lệnh kubectl apply hay restart service nào khác.

Bước 3 - Chờ kubelet tự khởi động lại static pod

```
sleep 60
```

Hướng dẫn kiểm tra lại

```
k get nodes # danh sách node hiển thị bình thường
```

```
k -n kube-system get pods | grep kube-apiserver # Running
```

```
sudo grep etcd-servers /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
# --etcd-servers=https://192.168.1.20:2379
```


Câu 15 — Expose a Node.js Deployment via NodePort

Đề bài

Reconfigure the existing Deployment `web-front` in namespace `svc-lab` to expose port `3000/tcp` of the existing container `node-app` (a Node.js application).

Tasks:

- Add the container port `3000/tcp` to the existing container `node-app` in the Deployment `web-front`.
- Create a new Service named `web-front-svc` exposing the container port `3000/tcp`.
- Configure the new Service to also expose the individual Pods via **NodePort** (let Kubernetes auto-assign the NodePort).

Lời giải

Bước 1 - Lấy YAML của Deployment hiện tại để chỉnh sửa

bash

```
kubectl get deployment web-front -n svc-lab -o yaml > deploy.yaml
```

Xuất ra file rồi sửa thay vì gõ tay lại toàn bộ Deployment - tránh làm sai lệch các field khác (labels, selector, replicas...) mà đề không yêu cầu đổi.

Bước 2 - Thêm containerPort vào đúng container

Mở `deploy.yaml`, thêm phần `ports` vào container `node-app`:

 **YAML**



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-front
  namespace: svc-lab
  labels:
    app: web-front

spec:
  replicas: 1

  selector:
    matchLabels:
      app: web-front

  template:
    metadata:
      labels:
        app: web-front

    spec:
      containers:
        - name: node-app
          image: node:20-alpine
          ports: # Thêm phần này
            - containerPort: 3000
```

Vì sao: dù Service vẫn hoạt động được nếu thiếu khai báo `containerPort` (miễn `targetPort` đúng), đề yêu cầu rõ phải khai báo port trên container

 <https://kubernetes.io/docs/concepts/services-networking/service/#field-spec-ports>

Bước 3 - Apply lại Deployment

```
kubectl apply -f deploy.yaml
```

Bước 4 - Expose Deployment bằng Service kiểu NodePort (không cố định port)

```
kubectl expose deployment web-front -n svc-lab --name web-front-svc --port 3000 --target-port 3000 --type NodePort
```

`kubectl expose` tự tạo Service dựa theo label selector có sẵn của Deployment, nhanh hơn nhiều so với viết YAML tay khi đề không yêu cầu gì đặc biệt như `nodePort` cố định. Vì không có yêu cầu port cố định, cứ để Kubernetes tự chọn ngẫu nhiên trong dải mặc định (30000-32767) là đủ. 

<https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport>

Hướng dẫn kiểm tra lại

```
kubectl describe deployment web-front -n svc-lab | grep -i port
```

```
# Port: 3000/TCP
```

```
kubectl get svc web-front-svc -n svc-lab # TYPE NodePort, PORT(S) 3000:3XXXX/TCP
```

```
kubectl get endpoints web-front-svc -n svc-lab # có IP pod, không phải <none>
```

```
curl http://\$NODE\_IP:\$NODE\_PORT # Trả kết quả hợp lệ
```

Câu 16 — Restrict an nginx Service to TLSv1.2 Only

Đề bài

There is a Deployment in the `secure-web` namespace using a ConfigMap `web-tls-cfg` that currently allows both TLSv1.2 and TLSv1.3, plus a Secret for the TLS certificate. A Service `secure-web` in the same namespace exposes it.

Tasks:

- Update the ConfigMap so only TLSv1.2 is accepted.
- Map the Service's ClusterIP to the hostname `tls-check.k8s.local` in `/etc/hosts`.
- Verify: `curl -vk --tlsv1.3 https://tls-check.k8s.local` must **fail**; `curl -vk --tls-max 1.2 https://tls-check.k8s.local` must **succeed**.
- Hoặc sửa configmap để chấp nhận cả `tlsv1.2` và `tlsv1.3`, và cấu hình configmap để nó immutable, còn phần map svc vào hostname trong `/etc/hosts` không có hỏi ạ.

```
kubectl patch configmap web-tls-cfg \
  -n secure-web \
  -p '{"immutable":true}'
```

Ví dụ:

❏ Bash

```
kubectl edit cm web-tls-cfg -n secure-web
```

sẽ báo lỗi tương tự:

```
ConfigMap "web-tls-cfg" is immutable
```

Muốn thay đổi nội dung phải:

❏ Bash

```
kubectl delete cm web-tls-cfg -n secure-web
kubectl apply -f cm.yaml
```

Lời giải

Bước 1 - Sửa ConfigMap, bỏ TLSv1.3

```
kubectl edit configmap web-tls-cfg -n secure-web
```

đổi: ssl_protocols TLSv1.2 TLSv1.3;

thành: ssl_protocols TLSv1.2;

 <https://kubernetes.io/docs/concepts/configuration/configmap/>

Bước 2 - Restart Deployment để pod đọc lại ConfigMap mới

```
kubectl rollout restart deployment secure-web -n secure-web
```

```
kubectl rollout status deployment secure-web -n secure-web
```

Vì sao: nếu ConfigMap được mount kiểu `subPath`, thay đổi ConfigMap sẽ **không** tự động sync vào container đang chạy (khác với mount thường) - bắt buộc phải rolling restart pod để nginx đọc lại file config mới. 

<https://kubernetes.io/docs/concepts/configuration/configmap/#mounted-configmaps-are-updated-automatically>

Hướng dẫn kiểm tra lại

```
curl -vk --tlsv1.3 https://tls-check.k8s.local
```

Kết quả mong đợi: TLS handshake failure (server không còn hỗ trợ TLSv1.3)

```
curl -vk --tls-max 1.2 https://tls-check.k8s.local
```

Kết quả mong đợi: HTTP/1.1 200 OK